

Deep Learning — Week 4 Notes

Feedforward Notation · Backpropagation · GD Variants (Batch / SGD / Mini-batch) · Momentum · AdaGrad · Adam

NPTEL / IIT Ropar · CS7015 Lecture 4 · Exam-focused revision sheet

1 · Feedforward Network — Notation

Layers, pre-activation, activation

A deep network stacks hidden layers then one output layer (the **L-th layer**). Each neuron splits into two parts: a **pre-activation** a_i (weighted sum) and an **activation** h_i (after the nonlinearity g).

$$a_i = W_i h_{i-1} + b_i \quad h_i = g(a_i) \quad h_L = \hat{y} = O(a_L)$$

- g = hidden activation function (logistic, tanh, linear ...).
- O = output activation function (softmax, linear ...).
- Parameters $\theta = (W_1, b_1, \dots, W_L, b_L)$. Forward pass computes every a_i , h_i , and $\hat{y}$.

Output + loss pairings

Output activation	Typical loss
Linear (regression)	Squared error
Softmax (classification)	Cross-entropy

The lecture focuses on **softmax output + cross-entropy loss**. Softmax turns scores into a probability distribution over k classes.

Softmax

$$\hat{y}_j = e^{a_{L,j}} / \sum_k e^{a_{L,k}}$$

Each output is in $(0,1)$ and all outputs sum to 1 — a valid probability distribution.

2 · Backpropagation (The Core Idea)

"Assigning responsibility" backward through the chain rule

To minimize loss we need $\nabla_{\theta} L$ — gradients w.r.t. every weight and bias. Backprop computes these efficiently: do one **forward pass**, then propagate the error **backward**, multiplying local derivatives (chain rule).

- **STEP 1** Output gradient: $\nabla_{a_L} L$ (how loss changes with output pre-activation).
- **STEP 2** For $k = L$ down to 1: gradient w.r.t. **parameters** (W_k, b_k), then w.r.t. the **layer below** (h_{k-1}, a_{k-1}).
- Intuition: the output layer "takes responsibility"; hidden layers share responsibility passed back through the weights above them.

$$\text{For softmax + cross-entropy: } \nabla_{a_L} L = (\hat{y} - y) \quad (\text{clean, no messy derivative})$$

Three families of gradients: (1) $\nabla_{a_L} L$ output layer; (2) $\nabla_{h_k} L, \nabla_{a_k} L$ hidden layers; (3) $\nabla_{W_k} L, \nabla_{b_k} L$ parameters. The chain rule "compresses" the multi-path product so shared work is reused.

Chain rule along multiple paths + activation derivative

If a variable influences the loss through several paths, its gradient is the **sum** over all paths. For hidden units you also multiply by the local activation slope $g'(a)$.

$$\text{logistic: } \sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad \tanh'(z) = 1 - \tanh^2(z)$$

3 · Gradient Descent Variants

How much data per update?

Method	Records per update	Trait
Batch GD	All N records	Stable, expensive
Stochastic GD	1 record	Noisy, cheap, fast
Mini-batch GD	Small batch (e.g. 3)	Balances noise & cost

Case Study 2 (12 records, batch size 3): mini-batches per epoch = $12 / 3 = 4$. One epoch = every record used once. Average of batch gradients (2,4,6,8) = 5. Mini-batch is preferred because it **balances noisy updates and computational efficiency**.

Case Study 1 — one-parameter GD step

$L(w) = (w - 4)^2$, so $dL/dw = 2(w - 4)$. At $w=1$, $\eta=0.1$.

Quantity	Computation	Result
dL/dw at $w=1$	$2(1-4)$	-6
next w	$1 - 0.1(-6)$	1.6
minimizer	$w - 4 = 0$	$w = 4$

w increases because the **gradient is negative**. The learning rate η **controls the step size** — it doesn't store targets or replace the gradient.

4 · Momentum

Smoothing the path with a running velocity

Successive gradients often point the same way but vary in size. **Momentum** keeps a running "velocity" so the current step depends on both the new gradient and part of the previous movement.

$$v_t = \beta v_{t-1} + \eta g_t \quad w_t = w_{t-1} - v_t$$

Case Study 3: $\beta=0.8$, $v_{t-1}=0.5$, $\eta=0.1$, $g_t=3$, $w_{t-1}=5$.

$v_t = 0.8(0.5) + 0.1(3) = 0.4 + 0.3 = \mathbf{0.7} \rightarrow w_t = 5 - 0.7 = \mathbf{4.3}$.

If $\beta=0$: $v_t = \eta g_t = 0.1(3) = \mathbf{0.3}$ (plain gradient step).

Meaning of β : the momentum term carries **part of the previous update direction**. Momentum can **smooth updates across steps** and speed up progress in consistent directions.

5 · Adaptive Learning Rates — AdaGrad & Adam

Per-parameter step sizes

Not all parameters get gradients of the same scale. **AdaGrad** accumulates squared gradients and shrinks the step for parameters that have received large gradients.

$$r_t = r_{t-1} + g^2 \quad \text{update magnitude} = \eta g / \sqrt{r_t}$$

Case Study 4: $g=4$, $r_{t-1}=9$, $\eta=0.5$.

$r_t = 9 + 4^2 = \mathbf{25}$ · $\sqrt{r_t} = \mathbf{5}$ · magnitude = $0.5(4)/5 = 2/5 = \mathbf{0.4}$.

- **AdaGrad idea:** adapts the step size using **accumulated squared gradients** — the step becomes smaller as r_t grows.
- **Adam:** combines **momentum-like averaging** (of gradients) **with adaptive scaling** (of squared gradients) — the best of both.

Exam quick-fire (all 20 answers)

- CS1: $dL/dw = -6$; next $w = 1.6$; w rises because gradient is negative; minimizer $w = 4$; η = step size.
- CS2: 4 mini-batches/epoch; batch GD uses all 12; SGD uses 1 record; $\text{avg}(2,4,6,8) = 5$; mini-batch balances noise & cost.
- CS3 momentum: $v_t = 0.7$; $w_t = 4.3$; β carries previous update direction; $\beta = 0 \rightarrow 0.3$; momentum smooths updates.
- CS4 AdaGrad: $r_t = 25$; $\sqrt{r_t} = 5$; magnitude = 0.4; adapts step via accumulated squared gradients; **Adam** = momentum + adaptive.
- Backprop: forward pass first; softmax+cross-entropy gives $\nabla_{a_L} L = \hat{y} - y$; multiply local derivatives backward.